6 Mapping URLs to endpoints using routing

This chapter covers

- Mapping URLs to endpoint handlers
- Using constraints and default values to match URLs
- Generating URLs from route parameters

In chapter 5 you learned how to define minimal APIs, how to return responses, and how to work with filters and route groups. One crucial aspect of minimal APIs that we touched on only lightly is how ASP.NET Core selects a specific endpoint from all the handlers defined, based on the incoming request URL. This process, called *routing*, is the focus of this chapter.

This chapter begins by identifying the need for routing and why it's useful. You'll learn about the endpoint routing system introduced in ASP.NET Core 3.0 and why it was introduced, and explore the flexibility routing can bring to the URLs you expose.

The bulk of this chapter focuses on the route template syntax and how it can be used with minimal APIs. You'll learn about features such as optional parameters, default parameters, and constraints, as well as how to extract values from the URL automatically. Although we're focusing on minimal APIs in this chapter, the same routing system is used with Razor Pages and Model-View-Controller (MVC), as you'll see in chapter 14.

In section 6.4 I describe how to use the routing system to *generate* URLs, which you can use to create links and redirect requests for your application. One benefit of using a routing system is that it decouples your handlers from